

# Dota 2 Trajectory Analyzer

## Compression MDL de trajectoires spatio-temporelles

Elias OKAT   Uzay TURKMENEL   Paul AUBERT

Université de Caen Normandie — L3 Informatique

Projet 2 — 2025-2026

# Contexte : DotA 2 et les données

**DotA 2** — MOBA compétitif, 2×5 joueurs

Chaque match  $\approx$  30-45 min  $\Rightarrow$  **flux** massif de  $(X, Y)$

Trois obstacles majeurs :

- ❶ **Volume** —  $\sim 10\text{M}$  pts / corpus
- ❷ **Bruit** — micro-déplacements
- ❸ **Pas de sémantique** — un point ne dit rien

## Question

Clics  $\rightarrow$  **comportements stratégiques** non supervisés ?



Carte officielle de DotA 2

# Pourquoi les approches existantes ne suffisent pas

## 1. Heatmaps statiques

- Montrent où les joueurs vont. . .
- Écrasent le temps : pas de séquence
- Jungle → Mid → Rivière ? Invisible.

## 2. Grilles manuelles

- Découpage arbitraire de la carte
- Biais humain aux frontières
- Ne s'adapte pas aux flux réels

## 3. Clustering direct sur points bruts

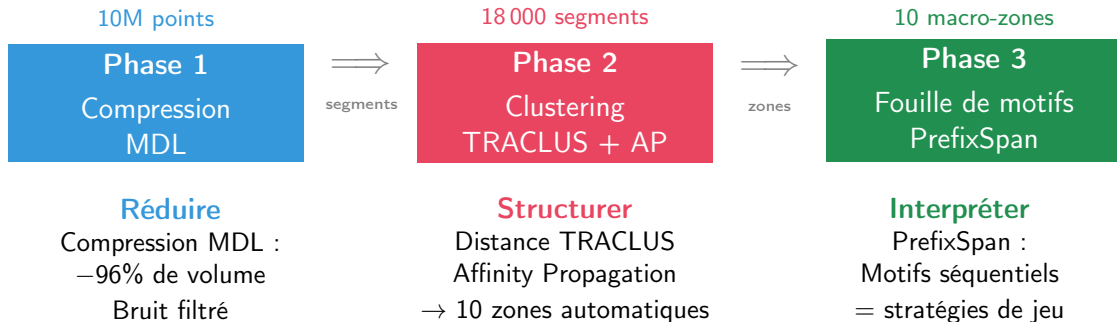
- Matrice  $N \times N$  :  $N = 10M \Rightarrow$   
**impossible**
- Le bruit noie le signal

### Constat

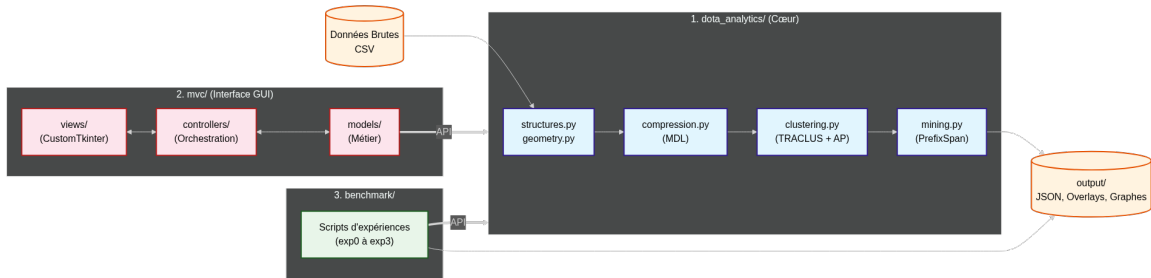
Il faut d'abord **compresser** les données pour rendre toute analyse possible.

⇒ Notre approche : un pipeline en 3 phases

# Notre pipeline en 3 phases



# Architecture logicielle — Diagramme de classes



## Pourquoi Python ?

- Écosystème **data science** mature
- Prototypage rapide, code lisible
- Interopérabilité C/Fortran via NumPy

## NumPy — calcul matriciel

- Matrices de distance  $N \times N$  (TRACCLUS)
- Vectorisation :  $\times 100$  vs boucles Python
- Mémoire efficace (float32)

## Matplotlib — visualisation

- Superposition trajectoires sur carte
- Graphiques multi-panneaux (benchmarks)

## Autres dépendances :

- `scikit-learn` — Silhouette, Davies-Bouldin
- `scipy` — Hausdorff, distances spatiales

## Idée clé (Minimum Description Length)

La description la plus courte qui reste fidèle à la trajectoire (erreur  $< w_{error}$ ).

### Algorithme glouton :

- 1 Partir du premier point
- 2 **Étendre** le segment point par point
- 3 Calculer  $d_{\perp}$  de chaque intermédiaire
- 4  $\max(d_{\perp}) < w_{error} \Rightarrow$  continuer ✓
- 5  $\max(d_{\perp}) \geq w_{error} \Rightarrow$  **couper** ✗
- 6 Reprendre au dernier point validé

$w_{error}$  = tolérance max :

- Petit  $\rightarrow$  haute fidélité, peu de compression
- Grand  $\rightarrow$  forte compression, perte de détail
- **Notre choix** :  $w_{error} = 12$

Complexité :  $\mathcal{O}(N^2)$  pire cas,  $\sim \mathcal{O}(N^{1.5})$  en pratique

## Résultat

Compression **96–98%** du volume, bruit filtré, erreur bornée.

## Résultat : avant / après compression



Trajectoire brute

Milliers de points, bruit inclus



Après MDL ( $w_{error} = 12$ )

~250 segments / joueur



# Données : du CSV brut au JSON compressé

```
tick,n1,n2,x0,y0,vec_x0,vec_y0,x1,y1,vec_x1,vec_y1,x2,y2,vec_x2,ve  
28290,0,1,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,  
28320,5,5,80.0,92.0,59.46875,71.15624,84.0,86.0,61.84375,51.34375,  
28350,5,5,80.0,94.0,67.6875,147.59375,84.0,88.0,154.46875,85.75,74
```



## Entrée — CSV brut

(x,y) toutes les 0,03 s → milliers de lignes

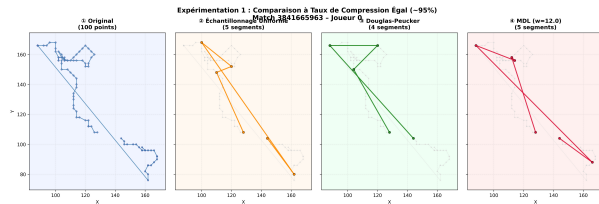
```
{  
  "match_id": "3841665963",  
  "w_error": 12.0,  
  "compression_info": {  
    "total_players": 10,  
    "timestamp": "2026-03-25T22:58:39.620842",  
    "algorithm": "MDL Greedy",  
    "formula": "Cost = L(H) + w_error * L(D|H)"  
  },  
  "players": [  
    {  
      "player_id": 0,  
      "num_segments": 75,  
      "num_original_points": 0,  
      "compression_rate": 0.0,  
      "segments": [  
        {  
          "start": {  
            "x": 80.0,  
            "y": 92.0,  
            "tick": 28320  
          },  
          "end": {  
            "x": 100.0,  
            "y": 168.0,  
            "tick": 29310  
          },  
          "length": 78.587530817554,  
          "angle": 75.25643716352927  
        },  
        ...  
      ]  
    },  
    ...  
  ]  
}
```

## Sortie — JSON compressé

Segments + labels de zones → exploitable par PrefixSpan

Le pipeline transforme un flux brut de coordonnées en **séquences symboliques** prêtes pour le *mining*.

# Exp. 1 — MDL vs Uniforme vs Douglas-Peucker



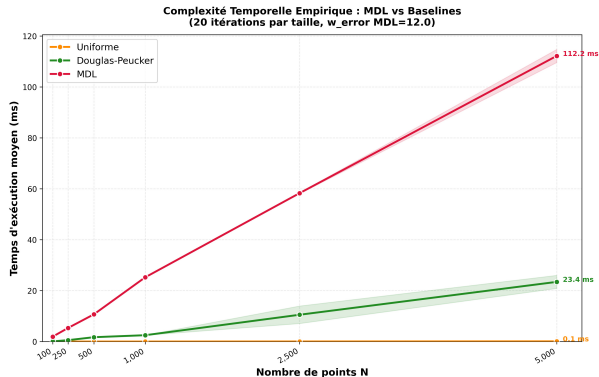
À compression égale ( $\sim 95\%$ ) :

| Méthode    | RMSE        | Hausd.       |
|------------|-------------|--------------|
| Uniforme   | 5.07        | 12.80        |
| Douglas-P. | 10.81       | 22.09        |
| <b>MDL</b> | <b>5.48</b> | <b>11.66</b> |

## Verdict

MDL : **meilleure fidélité** (Hausdorff min).  
Douglas-Peucker piégé par les outliers.

## Exp. 1b — Complexité temporelle : MDL vs Baselines



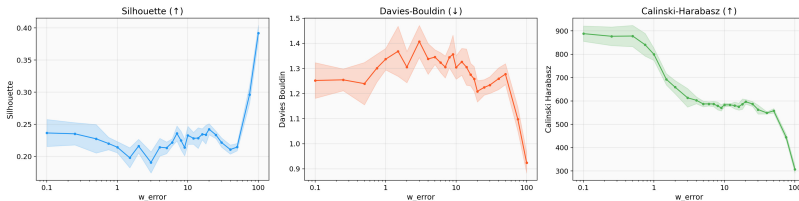
| Méthode    | Complexité              | 5 000 pts |
|------------|-------------------------|-----------|
| Uniforme   | $\mathcal{O}(1)$        | 0.1 ms    |
| Douglas-P. | $\mathcal{O}(N \log N)$ | 23 ms     |
| <b>MDL</b> | $\mathcal{O}(N^{1.5})$  | 112 ms    |

### Verdict

MDL plus lent mais **acceptable** ( $\sim 7s$  / match). Le coût se justifie par la **qualité**.

## Exp. 2 — Sensibilité de $w_{error}$ : pourquoi 12 ?

Fig 0.4 — Métriques de clustering vs  $w_{error}$



### Observation :

- $w_{error}$  petit  $\rightarrow$  fidélité mais faible compression
- $w_{error}$  grand  $\rightarrow$  plateau à  $\sim 96\%$
- **Coude** à  $w_{error} \approx 12$

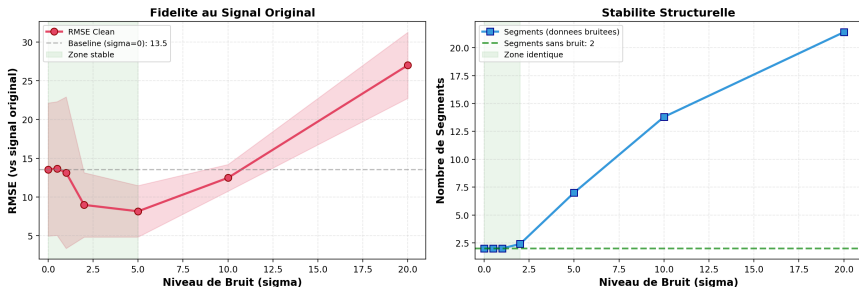
### Conclusion

$w_{error} = 12 =$  meilleur compromis  
compression / fidélité.

Silhouette se stabilise, bruit  $\sigma \leq 2$  absorbé.

## Exp. 3 — Robustesse de MDL au bruit gaussien

Robustesse MDL au Bruit Gaussien ( $w_{\text{error}} = 12$ )



**Protocole :** injection de bruit  $\mathcal{N}(0, \sigma^2)$  croissant sur 5 échantillons, compression MDL ( $w_{\text{error}} = 12$ ).

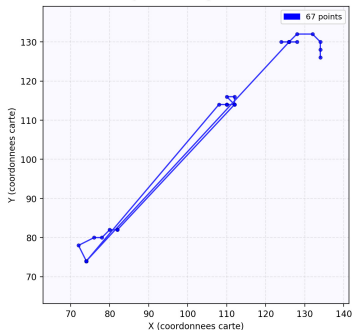
### Résultat

RMSE stable jusqu'à  $\sigma = 5$ .  
Structure identique ( $\sim 2$  seg.) jusqu'à  $\sigma = 2$ .  
MDL **filtre le bruit**, pas le signal.

# Exp. 3b — Visualisation : trajectoire propre → bruitée → filtrée

Robustesse de l'algorithme MDL face au bruit  
- Match 3841665963 - Joueur 0 - Ticks 66000-68000

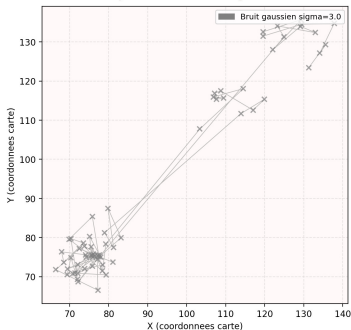
1. Trajectoire Originale (Clean)



1. Originale

67 points, trajectoire nette

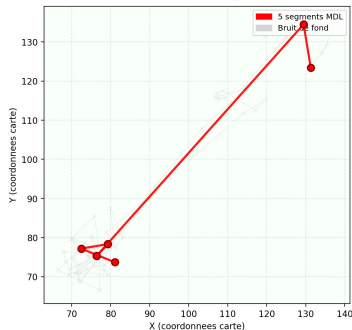
2. Injection de Bruit (sigma=3.0)



2. Bruit  $\sigma = 3$

Points dispersés, signal noyé

3. Resultat MDL (Filtrage w=12.0)

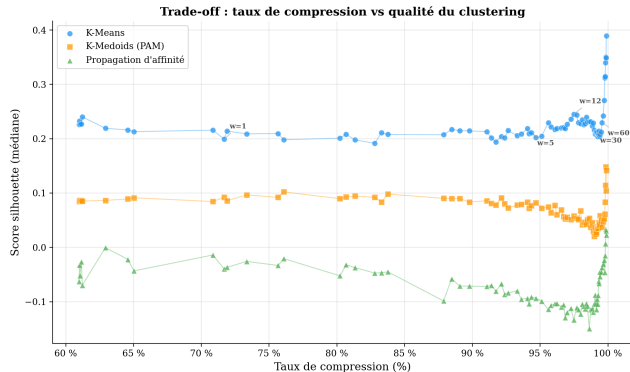


3. Après MDL

5 segments, structure retrouvée

MDL absorbe le bruit et **restitue la même structure** que la trajectoire propre.

# Compression → Clustering : quel impact ?



Chaque point = un  $w_{error}$  différent.  
 $w_{error} = 12$  annoté à  $\sim 96\%$  de compression.

## Conclusion

La compression MDL **ne dégrade pas** le clustering.  $w_{error} = 12$  est dans la zone stable.

# Évaluation comparative des algorithmes de clustering

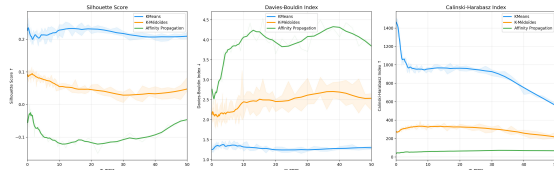
## Trois algorithmes en compétition :

- 1 K-Means (baseline euclidienne)
- 2 K-Médoïdes (PAM)
- 3 Affinity Propagation (AP)

## Avantage de la distance TRACLUS :

- K-Means limité à l'**euclidien brut**
- K-Médoïdes / AP exploitent la **matrice experte** (perp. + par. + angle)

Comparison KMeans / K-Médoïdes / Affinity Propagation



Scores de Silhouette par algorithme

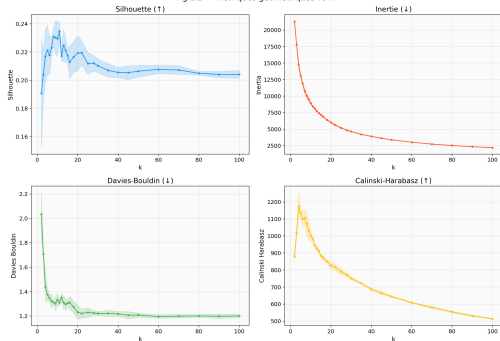
## Observation

K-Médoïdes et AP **doublent** le score Silhouette de K-Means grâce à TRACLUS.



# Optimisation de $k$ : compromis topologie / sémantique

Fig 1.3 — Métriques géométriques vs  $k$



Recherche du  $k$  optimal ( $k \in [2, 30]$ )

## Méthode du coude :

- **Davies-Bouldin** : minimum local à  $k \approx 10$  ✓
- **Silhouette** : stabilisation en plateau

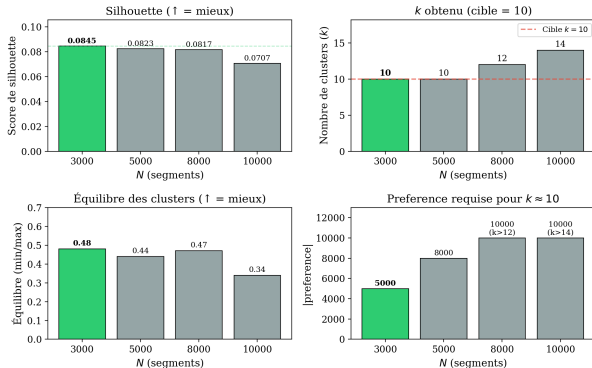
## Validation par PrefixSpan

À  $k = 10$ , volume de motifs **riche mais maîtrisé** : pas d'explosion combinatoire.

⇒ **Choix final** :  $k = 10$  zones

# Sensibilité de l'AP à la taille de l'échantillon

## Sensibilité d'Affinity Propagation au nombre de segments ( $N$ )



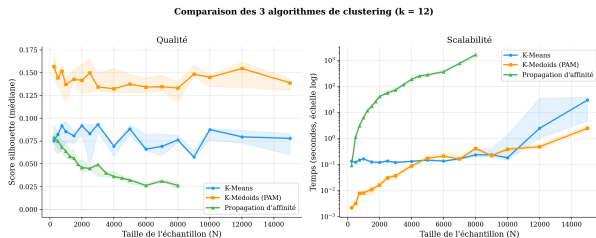
Vert = configuration optimale retenue ( $N=3000$ , preference=-5000,  $k=10$ ). Pour  $N \geq 8000$ , AP ne peut plus atteindre  $k=10$ .

- **Passage à l'échelle** : sur-segmentation (12–14 clusters) dès  $N \geq 8000$
- **Zone de convergence** :  $k = 10$  stable pour  $N = 3000$

## Décision technique

Sous-échantillonnage à  $N = 3000$  segments pour un découpage cohérent en 10 zones.

# Benchmark : qualité vs scalabilité ( $k = 12$ )



## Qualité (Silhouette)

- **K-Médoïdes** domine (médiane  $\approx 0.15$ )
- **AP** : instable si  $N > 8\,000$

## Scalabilité (temps)

- **K-Means / K-Médoïdes** : quasi linéaires
- **AP** :  $\mathcal{O}(N^2)$ , prohibitif  $> 4\,000$  seg.

Benchmark à  $k = 12$  fixé. En production, l'AP converge vers  $k = 10$ .

# Pourquoi l'AP malgré le K-Médoïdes ?



Les 10 macro-zones découvertes par l'AP

## Le paradoxe K-Médoïdes :

- Meilleur Silhouette, mais **doublons** : 2 centres à  $< 5$  unités
- Zones entières ignorées (Top Lane)
- Bon score  $\neq$  cohérence géographique

## Paramétrage AP retenu :

- $N = 3\,000$  segments
- Damping = 0.7
- Préférence =  $-5\,000$  ( $\rightarrow k \approx 10$ )

## Verdict

AP = seul algo avec **couverture complète** de la carte, servant d'**alphabet** pour PrefixSpan.

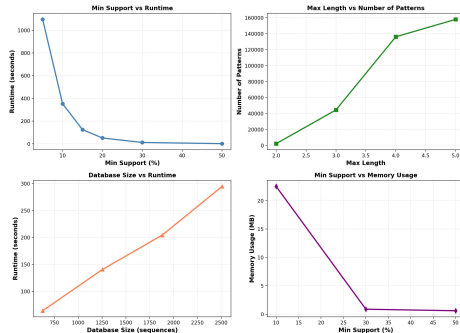
## Principe :

- Trouver les sous-séquences d'états fréquentes
- Approche par *pattern growth*

## Paramètres :

- minSupport :  $0.2 = 20\%$
- maxLength : 5

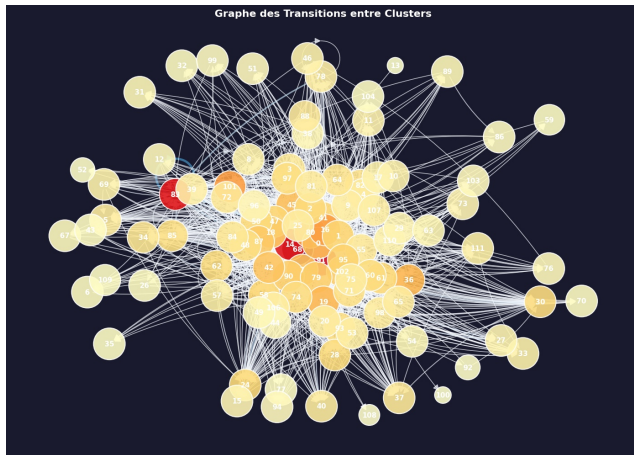
Benchmark PrefixSpan



# Résultats — Motifs et graphe de transitions

| Motif   | Sup. |
|---------|------|
| 14 → 68 | 22   |
| 91 → 68 | 24   |
| 91 → 36 | 21   |
| 68 → 83 | 19   |

**Interprétation :** Zones 14, 68, 91 = hubs stratégiques





Les 10 macro-zones découvertes par l'AP

## 3 stratégies émergentes :

### 1. Contrôle des runes ( $C8 \leftrightarrow C1 \leftrightarrow C7$ )

La rivière (C8) est le **hub central** : 2 400+ transitions vers mid. Les joueurs font des rotations constamment pour sécuriser les runes.

### 2. Corridor jungle ( $C4 \leftrightarrow C2 \leftrightarrow C9$ )

Circuit de *farming* : les joueurs enchaînent les camps de jungle en boucle (1 200+ transitions  $C4 \rightarrow C2$ ).

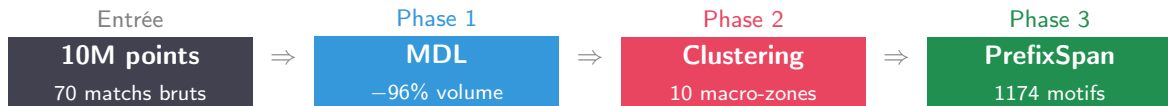
### 3. Isolement des bases (C3, C5)

Les zones de base (Radiant C3, Dire C5) sont **déconnectées** du graphe central — les joueurs n'y retournent qu'en early game ou en défense.

## Conclusion

Les stratégies émergent **sans supervision**, cohérentes avec la méta DotA 2.

# Bilan : de 10M de clics aux stratégies



## Résultats clés :

- Compression **96–98%** fidèle (erreur < 12 unités)
- Clustering AP : **10 zones** sans doublons ni trous
- Motifs séquentiels interprétables : cycles de runes, rotations jungle, corridors de transition

## Choix méthodologiques validés :

- Chaque paramètre justifié par benchmark
- AP et PrefixSpan codés *from scratch*
- 72 tests unitaires, graine fixée (SEED=42)
- Pipeline reproductible bout-en-bout



# Ce que l'on peut en tirer

## Enseignements techniques

- Un **bon score statistique** (Silhouette) ne garantit pas la **cohérence géographique**
- La compression est un **prérequis**, pas une option : sans MDL, aucun clustering n'est viable à cette échelle
- Le sous-échantillonnage maîtrisé ( $N=3\,000$ ) suffit pour capturer la structure globale

## Enseignements métier (DotA 2)

- Les **rotations de runes** et les **corridors mid→jungle** dominent les motifs fréquents
- Les stratégies émergent de la donnée **sans supervision** ni connaissance experte
- Le pipeline est **transposable** à d'autres jeux ou données spatio-temporelles

## Conclusion générale

De la donnée brute (18 000 segments) aux motifs interprétables (1 174 séquences), le pipeline a su **préserver et révéler une structure latente** que la seule observation visuelle ne permettait pas de saisir.

## Enrichissement des données

- **Étiquetage Radiant / Dire** : séquences synchrones par équipe pour extraire des stratégies multi-agents
- **Filtrage par rôle** (carry, support, mid...) : comparer les schémas de déplacement par poste
- **Événements de jeu** (kills, achats, tours) : relier les déplacements aux décisions stratégiques

## Améliorations algorithmiques

- **DBSCAN / HDBSCAN** : clustering sans fixer  $k$ , formes non sphériques
- **Fenêtre temporelle** : détecter les phases de jeu (early / mid / late game)
- **Compression adaptative** :  $w_{error}$  variable selon la densité locale

## Vision

Un outil d'analyse temps réel pour les équipes professionnelles et les casters.

# Merci de votre attention

Elias OKAT · Uzay TURKMENEL · Paul AUBERT

Université de Caen Normandie — L3 Informatique — Projet 2

Questions ?